

PROYECTO IS: MANUAL DE ESTÁNDARES Y ARQUITECTURA

1. ROLES DE LIDERAZGO TÉCNICO

Además de los roles habituales del equipo (Scrum Master, desarrolladores, QA), existen dos roles transversales, responsables durante todo el sprint (no solo en etapas puntuales):

1.1 Líder de Git

- Es el **primer filtro** de las PRs de su equipo: revisa que cumplan lo estipulado y las convenciones antes de pasar a revisión final.
- Mantiene las ramas del equipo organizadas (elimina ramas obsoletas/fusionadas, verifica nomenclatura).
- Mantiene las ramas actualizadas respecto a `dev` antes de cada PR.
- Es responsable de que la rama `epic` de su equipo se mantenga libre de conflictos.
- **Es el único autorizado para** revisar/aprobar PRs hacia la rama `epic`, y para enviar el PR de `epic` hacia `develop`.

1.2 Líder de Base de Datos

- Consolida los cambios de base de datos propuestos por los distintos equipos, evitando estructuras duplicadas o contradictorias.
- Verifica la correspondencia de nombres entre tablas/columnas de la BD y las entidades usadas en el backend.
- Cuida la integridad del esquema: ningún cambio estructural se aplica sin coordinarlo antes con los equipos afectados.

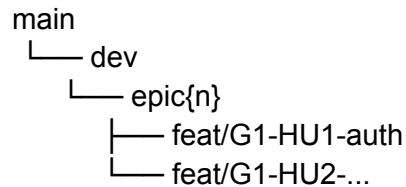
2. ORGANIZACIÓN DE RAMAS Y BASE DE DATOS COMPARTIDA

2.1 Ramas obligatorias del repositorio

- `main` — versión estable y desplegable. Solo se actualiza con integraciones ya validadas.
- `dev` — rama de integración principal. Todas las ramas de trabajo se integran aquí antes de pasar a `main`.

- **epic{n}** — rama intermedia asociada a una épica específica de un equipo; agrupa las historias de usuario relacionadas antes de integrarse a **dev**.
- **Rama de Historia de Usuario (HU)** — rama individual de historia de usuario con prefijo **feat/**

2.2 Jerarquía de integración



1. La rama de HU nace desde su **epic** correspondiente.
2. Al finalizar, se integra de vuelta a **epic** mediante PR **revisado y aprobado por el Líder de Git**.
3. Cuando todas las HU de una épica están completas, el **Líder de Git** integra **epic** en **develop** (es el único que puede enviar ese PR).
4. **develop** se integra en **main** una vez validada y estable.

2.3 Base de datos compartida entre equipos

- Existe una base de datos común a todos los equipos.
- Si un equipo necesita un cambio (nueva tabla, columna, relación), lo comunica en una **reunión exclusiva entre los Leads de Base de Datos** de cada equipo.
- La decisión de modificar la BD se toma de forma **consensuada** entre esos Leads, antes de continuar el desarrollo.

3. CONVENCIONES DE GIT

3.1 Nomenclatura de ramas (reemplaza el actual)

Tipo de rama	Formato	Ejemplo
Producción	main	main
Integración	dev	dev
Épica	epic{n}	epic1

Historia de Usuario	<code>feat/G{n}-HU{n}-nombreCorto</code>	<code>feat/G1-HU25-nombre</code>
---------------------	--	----------------------------------

- `epic{n}` lleva el número de la épica.
- `G{n}` identifica al grupo/equipo y `HU{n}` el número de historia de usuario.
- El nombre corto describe brevemente la funcionalidad (camelCase o palabras separadas por guiones): `auth`, `login`, `reporteVentas`.
- Todas las ramas de HU llevan el prefijo `feat/`, sea cual sea el tipo de cambio. El tipo real (`feat`, `fix`, `refactor`, `test`, `docs`) se indica en el commit (ver 3.2).

3.2 Convención de commits

Se usa Conventional Commits:

- **Formato:** `tipo(alcance): descripción`. Tipo y alcance van en minúscula; la descripción va en español, sin mayúscula inicial ni punto final.
- **Tipos:** `feat`, `fix`, `refactor`, `test`, `docs`, `chore` (según decisión 2).
- **Ejemplos:**
 - `feat(api): agregar endpoint de creación de empresas`
 - `fix(web): corregir validación de formulario de login`
 - `docs: agregar documentación de la API`
- **Commits inválidos:** el hook local rechaza el commit. Si llega a GitHub por otra vía, no se toma en cuenta.
- **Commits atómicos:** un cambio lógico por commit. Quien suba un commit con muchos cambios y el código falle, es responsable de corregirlo.

4. MANEJO DE DEPENDENCIAS ENTRE EQUIPOS

4.1 Uso de mocks

Al inicio del desarrollo, si un equipo depende de un endpoint o servicio de otro grupo que aún no está disponible, puede avanzar utilizando datos mockeados o valores temporales, sin quedar bloqueado.

4.2 Declaración de dependencias en la Fase 1

El sprint contempla dos fases de desarrollo (Fase 1: un subconjunto de historias de usuario; Fase 2: el resto de historias hasta completar el sprint). Aprovechando esa división:

- Durante la planificación de la Fase 1, cada equipo debe **anunciar explícitamente qué dependencias tiene con otros equipos** (qué necesita y de quién).

- El equipo del que se depende debe **priorizar** el desarrollo de esas dependencias dentro de su propia Fase 1, sin dejar de lado el resto de su backlog, pero dándoles preferencia frente a historias que no bloquean a nadie más.
- Al cierre de la Fase 1, los equipos dependientes **actualizan/sincronizan sus ramas** con lo ya integrado por el equipo proveedor, de modo que en la Fase 2 puedan completar su trabajo sin bloqueos pendientes.

4.3 Dependencias encadenadas entre varios equipos

Cuando la dependencia no es directa entre dos equipos sino que forma una cadena (un equipo depende de otro, que a su vez depende de un tercero, y así sucesivamente), declarar las dependencias de forma aislada no es suficiente: hace falta ver la cadena completa para priorizar correctamente. Para esto, se propone:

1. **Mapa de dependencias:** en la misma etapa de planificación de la Fase 1, se construye un mapa o matriz simple que muestre, para cada equipo, de qué otro equipo depende y en qué necesita.
2. **Orden de prioridad por cadena:** a partir de ese mapa, se identifica el orden en que deben resolverse las dependencias, los equipos que no dependen de nadie más (el inicio de cada cadena) priorizan primero lo que otros necesitan de ellos, de modo que la resolución se propague en cascada dentro del tiempo de la Fase 1.
3. **Coordinación conjunta:** este mapeo y orden se define en una reunión entre los Líderes de Git (o los responsables designados) de los equipos involucrados en la cadena, de forma análoga a la reunión de Leads de Base de Datos ya establecida.
4. **Si la cadena es demasiado larga para cerrarse en la Fase 1:** el equipo afectado continúa con mocks también en la Fase 2, pero debe comunicarlo de inmediato y actualizar su rama en cuanto la dependencia real esté lista, en lugar de esperar a un punto de sincronización fijo.

5. ESTÁNDARES GLOBALES DE CÓDIGO Y NOMENCLATURA

5.1 Lenguaje de desarrollo

- **Uso obligatorio de TypeScript:** todo el código del proyecto debe desarrollarse estrictamente en TypeScript. **Se exceptúan los archivos de configuración de herramientas.**
- **Excepción de JavaScript:** solo se aceptarán Pull Requests con archivos JavaScript en casos excepcionales, por incompatibilidad de bibliotecas o ausencia de tipos oficiales. Si existen definiciones de tipos (**@types**) disponibles, el PR con JavaScript será rechazado.

5.2 Idioma del código

- **Código técnico en inglés:** todo nombre de variable, función, clase, interfaz, archivo, carpeta y entidad de base de datos se escribe estrictamente en inglés.
- **Texto y documentación en español:**
 1. Interfaz de Usuario (UI): mensajes, etiquetas, botones, modales y alertas.
 2. Comentarios de código: explicaciones técnicas y documentación interna de lógica compleja.
 3. Ni un emoji en ningún lado, ni en la UI ni en comentarios.

5.3 Convenciones de formato

- **camelCase** (variables, funciones y métodos):
 - Funciones: iniciar con un verbo de acción (`getUserData`, `calculateTotal`, `handleFormSubmit`).
 - Booleanos: iniciar con prefijos de condición (`isOpen`, `hasError`, `canSubmit`).
- **PascalCase** (componentes, clases, tipos e interfaces):
 - Nombres representativos (`UserProfileCard`, `AuthService`).
 - Sufijos descriptivos obligatorios para contratos de datos (`Props`, `Response`, `Payload`, `Exception`).
- **kebab-case** (archivos y directorios): todo en minúsculas separado por guiones (`user-profile.tsx`, `auth-service.ts`).
- **UPPER_SNAKE_CASE** (constantes globales): para valores inmutables y configuraciones globales (`MAX_UPLOAD_SIZE`, `API_BASE_URL`).

5.4 Manejo de rutas dinámicas

- Formato con corchetes en minúsculas: `[id]`, `[slug]`, `[token]`. Se prioriza una sola palabra clara dentro del parámetro.

5.5 Patrón de acceso "Barril" (`index.ts`)

- Los archivos `index.ts/index.tsx` actúan **únicamente** como puentes de re-exportación.
- **Prohibición:** queda prohibido programar lógica de negocio, algoritmos o renderizado UI dentro de un archivo `index`.

5.6 Pruebas (nuevo)

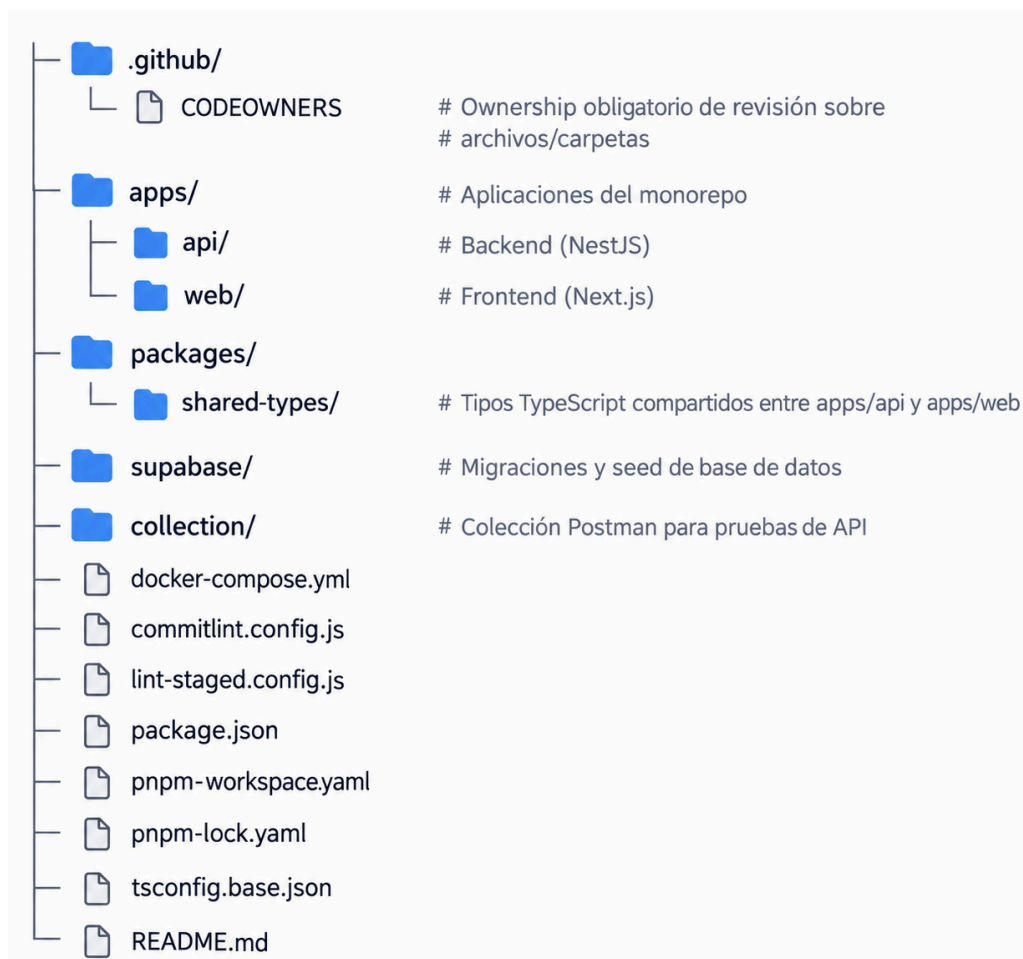
- **Backend:** Postman. Cada endpoint nuevo se agrega a `collection/`, en `companies.postman_collection.json` o en una colección nueva por módulo. El entorno local está en `collection/environments/local.postman_environment.json`.
- **Frontend:** Jest + Testing Library. Los tests van junto al archivo o en `__tests__/`.
Comando: `pnpm --filter web test`.

6. ARQUITECTURA Y ESTRUCTURA DEL REPOSITORIO

6.1 Visión general — Monorepo

El proyecto es un **monorepo gestionado con pnpm workspaces** (`pnpm-workspace.yaml`, `pnpm-lock.yaml`), con TypeScript compartido a nivel raíz (`tsconfig.base.json`).

Estructura raíz:



6.2 apps/api — Backend

- **Framework:** NestJS.
- **Configuración propia:** `nest-cli.json`, `tsconfig.json`, `package.json`.

Estructura interna (`src/`):

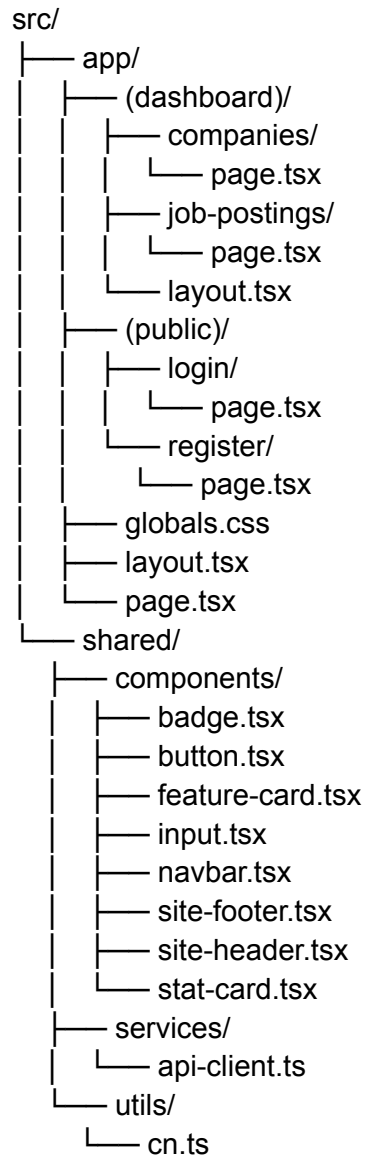


- **modules/** — organización por dominio de negocio (hasta ahora: `companies`, `job-postings`), cada uno con su propio `*.module.ts`.
- **shared/lib/** — integraciones transversales: cliente de **Redis** y cliente de **Supabase**.

6.3 apps/web — Frontend

- **Framework:** Next.js 14 (App Router), React 18.
- **Estilos:** TailwindCSS (+ `tailwind-merge`, `clsx`).
- **Testing:** Jest + Testing Library (`jest.config.js`, `jest.setup.js`).
- **Iconos:** `lucide-react`.

Estructura interna (**src/**):



- **app/** — rutas con grupos de rutas: (**dashboard**) para las pantallas autenticadas (**companies**, **job-postings**), (**public**) para **login/register**.
- **shared/components/** — componentes de UI reutilizables.
- **shared/services/api-client.ts** — cliente centralizado para llamadas a la API.
- **shared/utils/cn.ts** — utilidad (probablemente para combinar clases de Tailwind).

6.4 packages/shared-types

packages/shared-types/

```
|— src/
|   |— company.ts
|   |— job-posting.ts
|   |— index.ts
|— package.json
|— tsconfig.json
```

- Paquete interno del monorepo, consumido por `apps/api` y `apps/web` (aparece linkeado como `@umsspira/shared-types` en ambos).
- Contiene los tipos compartidos por dominio: `company.ts`, `job-posting.ts`, re-exportados desde `index.ts`.

6.5 supabase/ — Base de datos

supabase/

```
|— migrations/
|   |— 0001_create_companies.sql
|   |— 0002_create_job_postings.sql
|— seed.sql
```

- **Motor/BaaS:** Supabase.

6.6 Herramientas de calidad y automatización (raíz del monorepo)

- **.github/CODEOWNERS:** define quién debe aprobar obligatoriamente los cambios sobre ciertos archivos o carpetas del repositorio; GitHub exige la revisión del owner correspondiente antes de poder mezclar un PR que los modifique.
- **husky:** Git hooks que se ejecutan automáticamente en momentos clave (por ejemplo, antes de un commit o push), evitando que se suba código roto o mal formateado.
- **lint-staged.config.js:** corre linter/formateo únicamente sobre los archivos en staging, evitando que se suban archivos con código "basura" o no funcional. Se ejecuta típicamente disparado por un hook de husky.
- **commitlint.config.js:** valida que los mensajes de commit sigan el formato definido (sección 3.2: `feat`, `fix`, `refactor`, `test`, `docs`); si un commit no cumple el formato, es rechazado automáticamente.
- **docker-compose.yml:** entorno local.
- **collection/:** colección Postman (`companies.postman_collection.json`) y su entorno local, para probar los endpoints de `apps/api`.

7. ESTÁNDARES Y ARQUITECTURA DE BASE DE DATOS

7.1 Estándares generales

7.1.1 Roles de usuario mediante generalización

Se usará una tabla primaria `usuario` de la cual se extenderán las demás tablas de rol mediante generalización (herencia por FK 1:1 hacia `usuario`). Según el diagrama definido:

- `usuario` es la tabla base (`id`, `rol_id` FK hacia `rol`, `nombre`).
- Tablas que extienden a `usuario` mediante `usuario_id` (FK y PK a la vez): `mentor`, `empresa`, `egresado`, `administrador_de_evento`, `administrador_de_carrera`, `administrador_del_sistema`.
- `mentor` agrega además sus propios atributos (`nit`, `tamaño`).

7.1.2 Tipo de ID (PK)

- El tipo de cualquier `id` debe ser obligatoriamente **UUID**.
- El nombre de la columna PK siempre debe ser `id`.
- Al referenciar ese `id` desde otra tabla, la columna FK debe nombrarse `id_<nombre_tabla>` (ejemplo: `id_vacante`).

7.1.3 Columna de Estado

- No se deben crear tablas nuevas para representar estados.
- En su lugar, se añaden columnas con `checks` (ejemplo: `rechazado`, `aprobado`, `pendiente`, `observado`, etc.), según las necesidades de cada épica.

7.1.4 Columna de Nombre

- Toda columna de nombre debe ser de tipo `varchar(45)`.

7.2 Protocolos

7.2.1 Row Level Security (RLS)

- Todas las tablas con datos sensibles deben tener RLS habilitado.
- Las políticas se nombran en snake_case y minúsculas, con el patrón `tabla_accion_rol` (ejemplos: `mensajes_select propio`, `ofertas_delete_admin`).
- No se permite dejar una tabla sin políticas activas, aunque sea de solo lectura.

7.2.2 Autenticación y roles (JWT)

- El rol del usuario se define como un claim dentro del JWT (`app_metadata.role`) y se valida en cada política de RLS.
- Los valores de rol se escriben siempre en minúsculas y sin espacios (ejemplos: `egresado`, `administrador`, `reclutador`).

7.2.3 Nomenclatura de tablas, índices, funciones y triggers

Todo (funciones, triggers, tablas, índices) se nombra **en español**.

- **Tablas:** minúsculas, singular, `_` en vez de espacios. Ejemplos válidos: `oferta`, `usuario`, `categoria_producto`.
- **Índices:** formato obligatorio `idx_<tabla>_<columna>`, sin espacios, solo minúsculas. Ejemplo: `idx_oferta_precio`.
- **Funciones:** snake_case, minúsculas, deben describir la acción que realizan. Ejemplo: `fun_actualizar_timestamp`.
- **Triggers:** prefijo obligatorio `trg_`, seguido de tabla y evento que disparan. Formato: `trg_<tabla>_<evento>`. Ejemplo: `trg_oferta_before_update`.

7.2.4 Integridad de relaciones

- Cada registro relacionado con otra entidad debe mantener una relación válida (ejemplo: una postulación debe estar asociada a un egresado y a una oferta laboral existentes).
- No debe existir una postulación que referencie una oferta ya eliminada.
- Nunca deben existir tablas sueltas (sin relación) y no se deben borrar tablas de un esquema ya concebido.

7.2.5 Validación de información

- Antes de almacenar información, debe comprobarse que cumpla las reglas del sistema (ejemplo: una oferta laboral no debe aceptar salario negativo; el estado de una postulación debe usar valores definidos como `pendiente`, `aceptada` o `rechazada`).
- Se pueden aplicar `checks` (a evaluar cómo se implementan en cada caso).

7.2.6 Nulabilidad explícita

- Cada columna debe declararse explícitamente `NOT NULL`, salvo que exista una razón de negocio válida para permitir `NULL`.
- Justificación: los `NULL` rompen operaciones matemáticas, funciones de agregación (`COUNT`, `SUM`), concatenaciones de texto y sentencias lógicas si no se manejan explícitamente. Si un dato es obligatorio para el negocio, la base de datos debe rechazar el insert si viene vacío.

7.2.7 Atomicidad de datos

- Cada columna debe almacenar un único dato atómico. Prohibido guardar listas, arrays separados por comas o JSONs complejos en una columna estándar si ese dato debería ser una relación normalizada.
- Ejemplo incorrecto: columna `telefonos` con `"555-1234, 555-5678"`.
- Ejemplo correcto: tabla relacionada `telefono` con `usuario_id`.
- Justificación: violar la atomicidad impide filtrar, buscar, ordenar o validar datos individuales de forma estándar en SQL.

7.2.8 Documentación de tablas, triggers, columnas y funciones

- Es obligatorio, al crear una tabla, agregar comentarios indicando para qué es y con qué tablas estará relacionada.
- Lo mismo aplica a funciones: comentarios indicando para qué son y con qué tablas u otras funciones se relacionan.
- Se exceptúan columnas generales muy evidentes (`id`, `nombre`, `email`), salvo que sea necesario.

7.2.9 Control de duplicados

- Los datos que deben ser únicos no pueden registrarse más de una vez (ejemplo: un egresado no debe poder crear dos cuentas con el mismo correo).
- Se debe implementar una subconsulta de validación (`unique(fun_existe(algo)? true/false)`).
- Cada función en el backend que realice registro, eliminación, etc., debe manejar transacciones (`commit/rollback`).

7.2.10 Auditoría y trazabilidad de cambios (Audit Logging)

- Registra automáticamente el historial completo de toda modificación, creación o eliminación de datos: fecha, hora y usuario que realizó la acción.
- Se implementa mediante una tabla de auditoría (`id`, `nombre`, `función`, `hora`).

7.2.11 Respallos y recuperación en el tiempo (PITR / Backups)

- Copias de seguridad continuas que permiten restaurar la base de datos a un estado exacto en cualquier segundo del pasado reciente ante un fallo crítico.

7.2.12 Connection Pooling

- Se usará un protocolo de connection pooling para controlar y reutilizar conexiones a la base de datos (patrón cliente → PgBouncer → pool → PostgreSQL).
- Beneficio: conexiones concurrentes y uso optimizado de recursos.

- Cómo se usa: en configuraciones de Supabase, buscar "connection pooling" y seleccionar **session mode**; luego revisar documentación específica según la tecnología del stack que consuma la conexión.

7.3 Métricas

Métrica	Objetivo / definición
Tiempo promedio de consulta	Ej.: búsqueda de ofertas laborales debe responder en menos de 800 ms
Disponibilidad de la base de datos	Objetivo: 99% de disponibilidad mensual
Tasa de conexiones rechazadas	Objetivo: ≤ 1 conexión rechazada por hora
Tasa de registros duplicados	Ej.: % de intentos de registro con correo ya existente, medido mensualmente
Conexiones y sesiones (connection churn)	Mide qué tan seguido se abren/cierran conexiones; con Prisma se evita reutilizando el mismo PrismaClient en toda la app en vez de instanciarlo por request
Transacciones y bloqueos (lock wait time)	Tiempo que una transacción espera a que otra libere un lock; monitoreable vía pg_locks ; valores altos indican transacciones muy largas o falta de índices
Almacenamiento y crecimiento (WAL pendientes)	Archivos WAL generados pero no archivados/aplicados en el standby; si crece sin control, el disco puede llenarse y Postgres bloquear escrituras nuevas